Later in this chapter, we shall introduce constraints other than functional dependencies. We say that a relation is **legal** if it satisfies all rules, or constraints, that we impose on our database.

Let *C* represent a set of constraints on the database, and let R be a relation schema. A decomposition $\{R_1, R_2, \ldots, R_n\}$ of R is a **lossless-join decomposition** if, for all relations *r* on schema *R* that are legal under *C*,

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \cdots \bowtie \Pi_{R_n}(r)$$

We shall show how to test whether a decomposition is a lossless-join decomposition in the next few sections. A major part of this chapter deals with the questions of how to specify constraints on the database, and how to obtain lossless-join decompositions that avoid the pitfalls represented by the examples of bad database designs that we have seen in this section.

7.5 Desirable Properties of Decomposition

We can use a given set of functional dependencies in designing a relational database in which most of the undesirable properties discussed in Section 7.2 do not occur. When we design such systems, it may become necessary to decompose a relation into several smaller relations.

In this section, we outline the desirable properties of a decomposition of a relational schema. In later sections, we outline specific ways of decomposing a relational schema to get the properties we desire. We illustrate our concepts with the *Lending-schema* schema of Section 7.2:

Lending-schema = (*branch-name, branch-city, assets, customer-name, loan-number, amount*)

The set *F* of functional dependencies that we require to hold on *Lending-schema* are

branch-name → *branch-city assets*
loan-number → *amount branch-name*

As we discussed in Section 7.2, *Lending-schema* is an example of a bad database design. Assume that we decompose it to the following three relations:

Branch-schema = (*branch-name, branch-city, assets*)
Loan-schema = (*loan-number, branch-name, amount*)
Borrower-schema = (*customer-name, loan-number*)

We claim that this decomposition has several desirable properties, which we discuss next. Note that these three relation schemas are precisely the ones that we used previously, in Chapters 3 through 5.

7.5.1 Lossless-Join Decomposition

In Section 7.2, we argued that, when we decompose a relation into a number of smaller relations, it is crucial that the decomposition be lossless. We claim that the

decomposition in Section 7.5 is indeed lossless. To demonstrate our claim, we must first present a criterion for determining whether a decomposition is lossy.

Let R be a relation schema, and let F be a set of functional dependencies on R. Let R_1 and R_2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if at least one of the following functional dependencies is in F^+:

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

In other words, if $R_1 \cap R_2$ forms a superkey of either R_1 or R_2, the decomposition of R is a lossless-join decomposition. We can use attribute closure to efficiently test for superkeys, as we have seen earlier.

We now demonstrate that our decomposition of *Lending-schema* is a lossless-join decomposition by showing a sequence of steps that generate the decomposition. We begin by decomposing *Lending-schema* into two schemas:

Branch-schema = *(branch-name, branch-city, assets)*
Loan-info-schema = *(branch-name, customer-name, loan-number, amount)*

Since *branch-name* $\rightarrow$ *branch-city assets*, the augmentation rule for functional dependencies (Section 7.3.2) implies that

branch-name $\rightarrow$ *branch-name branch-city assets*

Since *Branch-schema* $\cap$ *Loan-info-schema* $= \{$*branch-name*$\}$, it follows that our initial decomposition is a lossless-join decomposition.

Next, we decompose *Loan-info-schema* into

Loan-schema = *(loan-number, branch-name, amount)*
Borrower-schema = *(customer-name, loan-number)*

This step results in a lossless-join decomposition, since *loan-number* is a common attribute and *loan-number* $\rightarrow$ *amount branch-name*.

For the general case of decomposition of a relation into multiple parts at once, the test for lossless join decomposition is more complicated. See the bibliographical notes for references on the topic.

While the test for binary decomposition is clearly a sufficient condition for lossless join, it is a necessary condition only if all constraints are functional dependencies. We shall see other types of constraints later (in particular, a type of constraint called multivalued dependencies), that can ensure that a decomposition is lossless join even if no functional dependencies are present.

7.5.2 Dependency Preservation

There is another goal in relational-database design: *dependency preservation*. When an update is made to the database, the system should be able to check that the update will not create an illegal relation—that is, one that does not satisfy all the given

functional dependencies. If we are to check updates efficiently, we should design relational-database schemas that allow update validation without the computation of joins.

To decide whether joins must be computed to check an update, we need to determine what functional dependencies can be tested by checking each relation individually. Let F be a set of functional dependencies on a schema R, and let $R_1, R_2, \ldots, R_n$ be a decomposition of R. The **restriction** of F to R_i is the set F_i of all functional dependencies in F^+ that include *only* attributes of R_i. Since all functional dependencies in a restriction involve attributes of only one relation schema, it is possible to test such a dependency for satisfaction by checking only one relation.

Note that the definition of restriction uses all dependencies in F^+, not just those in F. For instance, suppose $F = \{A \rightarrow B, B \rightarrow C\}$, and we have a decomposition into AC and AB. The restriction of F to AC is then $A \rightarrow C$, since $A \rightarrow C$ is in F^+, even though it is not in F.

The set of restrictions $F_1, F_2, \ldots, F_n$ is the set of dependencies that can be checked efficiently. We now must ask whether testing only the restrictions is sufficient. Let $F' = F_1 \cup F_2 \cup \cdots \cup F_n$. F' is a set of functional dependencies on schema R, but, in general, $F' \neq F$. However, even if $F' \neq F$, it may be that $F'^+ = F^+$. If the latter is true, then every dependency in F is logically implied by F', and, if we verify that F' is satisfied, we have verified that F is satisfied. We say that a decomposition having the property $F'^+ = F^+$ is a **dependency-preserving decomposition**.

Figure 7.12 shows an algorithm for testing dependency preservation. The input is a set $D = \{R_1, R_2, \ldots, R_n\}$ of decomposed relation schemas, and a set F of functional dependencies. This algorithm is expensive since it requires computation of F^+; we will describe another algorithm that is more efficient after giving an example of testing for dependency preservation.

We can now show that our decomposition of *Lending-schema* is dependency preserving. Instead of applying the algorithm of Figure 7.12, we consider an easier alternative: We consider each member of the set F of functional dependencies that we

```
compute F+;
for each schema Ri in D do
    begin
        Fi := the restriction of F+ to Ri;
    end
F' := ∅
for each restriction Fi do
    begin
        F' = F' ∪ Fi
    end
compute F'+;
if (F'+ = F+) then return (true)
                  else return (false);
```

Figure 7.12 Testing for dependency preservation.

require to hold on *Lending-schema*, and show that each one can be tested in at least one relation in the decomposition.

- We can test the functional dependency: *branch-name* → *branch-city assets* using *Branch-schema* = (*branch-name*, *branch-city*, *assets*).
- We can test the functional dependency: *loan-number* → *amount branch-name* using *Loan-schema* = (*branch-name*, *loan-number*, *amount*).

If each member of F can be tested on one of the relations of the decomposition, then the decomposition is dependency preserving. However, there are cases where, even though the decomposition is dependency preserving, there is a dependency in F that cannot be tested in any one relation in the decomposition. The alternative test can therefore be used as a sufficient condition that is easy to check; if it fails we cannot conclude that the decomposition is not dependency preserving, instead we will have to apply the general test.

We now give a more efficient test for dependency preservation, which avoids computing F^+. The idea is to test each functional dependency $\alpha \rightarrow \beta$ in F by using a modified form of attribute closure to see if it is preserved by the decomposition. We apply the following procedure to each $\alpha \rightarrow \beta$ in F.

result = α
while (changes to *result*) **do**
  for each R_i in the decomposition
    t = (*result* $\cap R_i$)$^+$ $\cap$ R_i
    result = *result* $\cup$ t

The attribute closure is with respect to the functional dependencies in F. If *result* contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved. The decomposition is dependency preserving if and only if all the dependencies in F are preserved.

Note that instead of precomputing the restriction of F on R_i and using it for computing the attribute closure of *result*, we use attribute closure on (*result* $\cap R_i$) with respect to F, and then intersect it with R_i, to get an equivalent result. This procedure takes polynomial time, instead of the exponential time required to compute F^+.

7.5.3 Repetition of Information

The decomposition of *Lending-schema* does not suffer from the problem of repetition of information that we discussed in Section 7.2. In *Lending-schema*, it was necessary to repeat the city and assets of a branch for each loan. The decomposition separates branch and loan data into distinct relations, thereby eliminating this redundancy. Similarly, observe that, if a single loan is made to several customers, we must repeat the amount of the loan once for each customer (as well as the city and assets of the branch) in *lending-schema*. In the decomposition, the relation on schema *Borrower-schema* contains the *loan-number*, *customer-name* relationship, and no other schema does. Therefore, we have one tuple for each customer for a loan in only the relation